

Information Processing 1 — Week 15 exercises

Projects can be submitted the same week or the following week with no penalty.

New projects will be added each week. Each project is worth a certain number of points. You can score up to 50 points in total from all the projects you complete.

When you finish a project, **immediately** ask an instructor to grade it. If you are stuck or have any questions then **raise your hand** and an instructor will help you.

1 Object-oriented programming: gas simulation [6 points]

Just as Monte Carlo simulations using random numbers can discover numerical values, physical simulations can find or confirm laws describing how the world works. One such simulation is of an ideal gas involving molecules bouncing off the walls of a closed container.

1.1 Task

- create a class `Sprite` as a *subclass* of `Turtle`
- add an `__init__(self, x, y, heading, speed)` method to `Sprite` that turns off drawing, sets its shape to `"circle"`, sets its position, and its velocity vector
- add a `step(self)` method to `Sprite` that updates its position based on its velocity and, if it hits a wall, adjusts the velocity to make it “bounce” off the wall
- create 100 `Sprites` and repeatedly `step` them to simulate the motion of gas molecules in a container

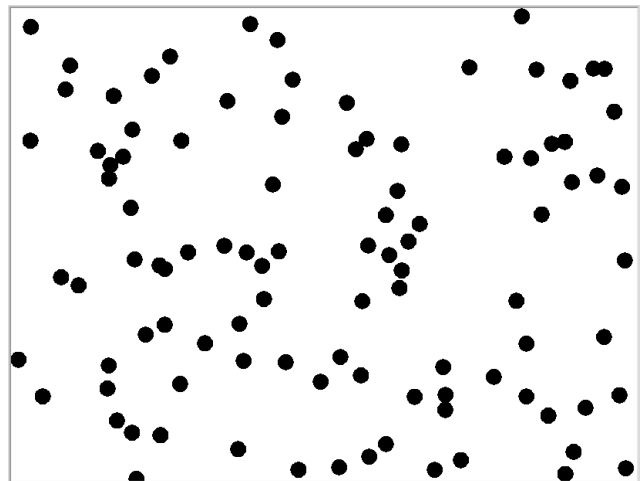
1.2 Example session

There is no user interaction. When run, the program displays 100 ‘atoms’ that move around inside the window, bouncing off the sides.

1.3 Program structure

The program structure can be very simple, consisting of only four parts.

1. The `import` declarations for modules and functions.
2. The definition of the class `Sprite` and its two methods `__init__` and `step`.
3. A loop that creates 100 `Sprites` and stores them in a list.
4. An infinite loop that iterates over the list of `Sprites` and tells each one to `step` itself.



1.4 Creating a subclass

A normal class begins completely 'empty' and you have to implement everything it does. A subclass is based on an existing class and extends it with only the additional features that are needed.

The syntax to create a subclass is

```
class Subclass(BaseClass):
    ... # additional methods
```

which creates `Subclass` as a subclass of `BaseClass`.

One subtle point about subclasses is that it is your responsibility to call the `__init__` method of the base class. You can omit it if there is no special initialisation needed in the base class, but if you do need special initialisation then in the above example you might add the following method:

```
class Subclass(BaseClass):
    def __init__(self):
        BaseClass.__init__(self) # initialise the base class part of self
        ...                     # initialise the subclass part of self
```

1.5 Turning a turtle into a sprite

In the `Sprite.__init__` method you can set up `self` (which is already a kind of `Turtle` because of the base class) to switch off drawing and change shape to a circle.

```
self.up()           turns off drawing
self.goto(x, y)     sets the initial position
self.shape("circle") changes the shape to a circle
```

You might also add two attributes to `self` to hold the velocity vector; in other words, the amounts by which the `x` and `y` positions should change each time the `Sprite` is stepped.

In the `Sprite.step` method you can obtain the current position using the inherited (from `Turtle`) method `self.position()` which tells you the current position as a `tuple`.

To make the `Sprite` move, add the velocity vector's horizontal and vertical components to the position and then use `self.goto(newX, newY)` to update the position.

To make the `Sprite` bounce off walls, check to see if the new position is within some boundary distance (10 pixels, for example) of each of the walls and negate the horizontal component of the velocity vector for a 'collision' with one of the side walls or the vertical component for a collision with the top or bottom wall.

1.6 Program initialisation

You can store your 100 `Sprites` in a list. For each `Sprite` you create, set the `x` and `y` values randomly to lie somewhere within the window (making sure to avoid the boundary distance to the walls), and the heading to some random angle, and the speed to some random speed in a reasonable range (such as 3 to 7).

1.7 Stepping the simulation

This is as easy as iterating over the list of `Sprites` you created in the previous step and asking each one of them to `.step()` itself.

1.8 Improving performance

As with previous turtle-based graphics programs you have written, you can drastically improve performance by turning off automatic updates and the corresponding delay. Assuming you did

```
from turtle import *
```

at the start of your program, you can add

```
tracer(0, 0)
```

before the main loop, and then call

```
update()
```

from your main loop immediately after the `step()` ping loop has finished.

A Challenge: determine a physical law by simulation

You can estimate the pressure on the 'walls' of the window 'box' by summing the change in momentum of the particles as they bounce off each wall. Divide the total by the number of completed world updates to obtain an estimate of the pressure. This number should stabilise rapidly as the number of update cycles increases.

Next you can change the number N of molecules, perhaps from 50 to 500 in increments of 50. Plot the graph of P versus pressure N to determine how P varies with N .

You can then change the average velocity V of the molecules, perhaps from 5 to 50 in increments of 5. Plot the graph of P versus pressure V to determine how P varies with V .

Combine the way N and V affect P , with some constant of proportionality, and check whether you have obtained the 'ideal gas equation'.